Bachelor of Science in Computer Science & Engineering

Infra Monitor

Development Lab Project Report

Team Members

Syed Fardin Ahmed
ID: 2204014

Shahneos Ali
ID: 2204013

Ahmed Abrar Zayad
ID: 2204021

Department of Computer Science and Engineering
Chittagong University of Engineering and Technology (CUET)
Chattogram-4349, Bangladesh

November 2026

Chittagong University of Engineering & Technology (CUET)
Department of Computer Science & Engineering
Chattogram-4349, Bangladesh

Development Lab Project Report

Infra Monitor

Course Code	CSE-356
Course Title	Development Lab [TO CONFIRM]
Submitted to	Mir Md. Saki Kowser Md. Refaj Hossan
Department	Computer Science and Engineering
Institution	Chittagong University of Engineering and Technology
Semester / Session	[TO CONFIRM]
Submission Date	[TO CONFIRM]

Student	**Reported contribution**
Syed Fardin Ahmed (2204014)	Authentication, SMTP, role-based access control, and request sanitization
Shahneos Ali (2204013)	User interface and notification service
Ahmed Abrar Zayad (2204021)	Remaining project features, as confirmed by the team

Contents

1 Introduction **1**

1.1 Overview . . . 1

1.2 Problem Statement . . . 1

1.3 Objectives . . . 1

1.4 Scope and Organization . . . 2

2 System Architecture and Design **3**

2.1 System Overview . . . 3

2.2 Technology Stack . . . 4

2.3 System Design Diagrams . . . 4

2.3.1 Entity–Relationship Model . . . 4

2.3.2 Use Cases . . . 5

2.4 Database Schema . . . 5

2.5 User Interface Design . . . 6

3 Monitoring and Intelligent Features **8**

3.1 Telemetry Collection . . . 8

3.2 Anomaly Detection and Request Shield . . . 8

3.3 Advisory Assistant . . . 8

3.4 Operational Workflows . . . 9

4 Implementation Details **10**

4.1 Application Modules . . . 10

4.2 Selected API Surface . . . 10

4.3 Security and Access Control . . . 11

4.4 Local Execution and Verification . . . 11

5 Conclusion **12**

Chapter 1

Introduction

1.1 Overview

Infra Monitor is a project for registering Linux hosts and presenting their operational telemetry and events in one interface. The repository describes a Django backend, a Flutter client, and a monitoring pipeline built around Grafana Alloy and VictoriaMetrics. The product groups monitoring work by organization, server, and service, with dashboards, alerts, incidents, logs, and anomaly records exposed through the application ([1]; `backend/infra_monitor/organization_urls.py`).

1.2 Problem Statement

Host and service health signals are useful only when operators can associate them with the correct environment and act on resulting alerts. In this project, the backend maintains organization-scoped server and service records while time-series samples are queried from VictoriaMetrics. The design therefore addresses the integration problem of collecting telemetry, retaining operational context, and presenting it alongside incident workflows (`docs/monitoring-end-to-end.md`; `backend/servers/models/servers.py`; `backend/incident/models/incident.py`).

1.3 Objectives

1. Provide authenticated organization and membership workflows.
2. Enroll monitored servers and forward telemetry through a controlled ingestion path.
3. Present server/service health, metrics, alerts, incidents, and anomaly detections in the Flutter client.
4. Integrate machine-learning services for telemetry anomaly inference and HTTP request classification, while keeping their outputs distinguishable from operator-confirmed state.

These objectives describe implemented repository areas, not measured operational outcomes. The report does not claim production availability, performance benchmarks, or user-study results.

1.4 Scope and Organization

This report focuses on the current source tree and supplied interface screenshots. It summarizes system design, the telemetry and ML workflows, and the implementation structure. It does not treat the unrelated ClubHub content in the reference report as project evidence, nor infer an actual semester schedule, budget, or deployment outside the local Compose configuration. Chapter 2 describes the architecture and core data model; Chapter 3 explains telemetry and intelligent features; Chapter 4 covers implementation and available verification commands; Chapter 5 summarizes limitations and future work.

Chapter 2

System Architecture and Design

2.1 System Overview

The Flutter client communicates with organization-scoped Django API routes. Django coordinates authentication, domain records, metric reads and writes, and background tasks. PostgreSQL stores the operational control-plane data; VictoriaMetrics stores time-series data; Redis supports Celery; and the FastAPI service provides ML endpoints. Docker Compose defines these components for local execution ([2]; `backend/infra_monitor/urls.py`; `backend/infra_monitor/organization_urls.py`).

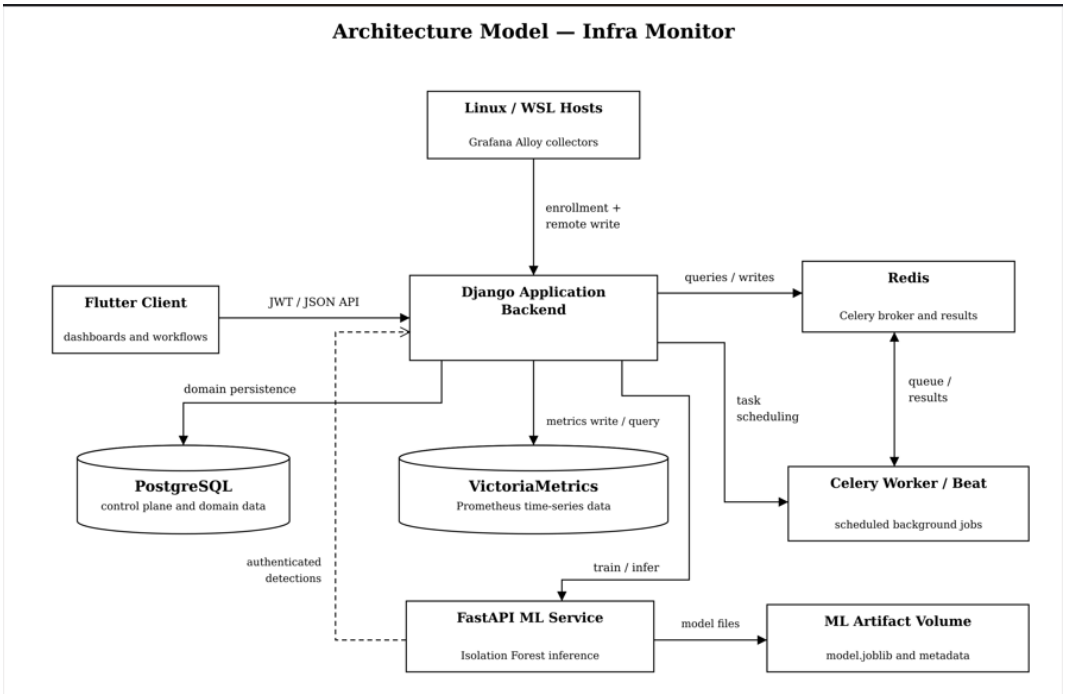


Figure 2.1: Supplied Infra Monitor architecture diagram. The current FastAPI service also exposes Request Shield classification, and the Gemini assistant is implemented through Django; these paths are not drawn in this source figure.

The figure is retained as a useful high-level view, but it is not a complete deployment diagram. The model service's current routes are defined in `model/app/main.py`; the conversational assistant's provider call is in `backend/ai/services.py`. This distinction avoids implying that every AI feature is hosted by the same model endpoint.

2.2 Technology Stack

Table 2.1: Principal technologies evidenced in the repository

Area	Technology	Repository evidence / role
Client	Flutter / Dart	Cross-platform interface; dependencies are declared in `frontend/pubspec.yaml`.
API and application	Django, Django REST Framework, Channels	Organization, monitoring, and workflow routes; dependency names are listed in `backend/requirements.txt`.
Operational store	PostgreSQL	Relational application data, configured as a Compose service.
Time-series store	VictoriaMetrics cluster	Stores/query source for Prometheus-compatible telemetry in `docker-compose.yml`.
Background work	Celery and Redis	Worker, beat scheduler, broker, and results services in `docker-compose.yml`.
ML API	FastAPI, scikit-learn	Inference/training and request classification in `model/app/main.py` and `model/requirements.txt`.
Host collection	Grafana Alloy	Installed on monitored Linux hosts; remote-write flow is documented in `docs/monitoring-end-to-end.md`.

Exact versions are stated only where pinned by manifests or image tags; the Python requirements intentionally do not pin every package version.

2.3 System Design Diagrams

2.3.1 Entity–Relationship Model

The following black-and-white conceptual ER view uses entity boxes and relationship diamonds. It focuses on the main organization, membership, and monitoring records rather than drawing every supporting table. Membership is an entity because it stores role and approval data. The relationships and keys shown are based on Django models in `backend/accounts/models/organization_membership.py`, `backend/servers/models/servers.py`, `backend/servers/models/service.py`, and `backend/ml_model/models/anomaly_detection.py`.

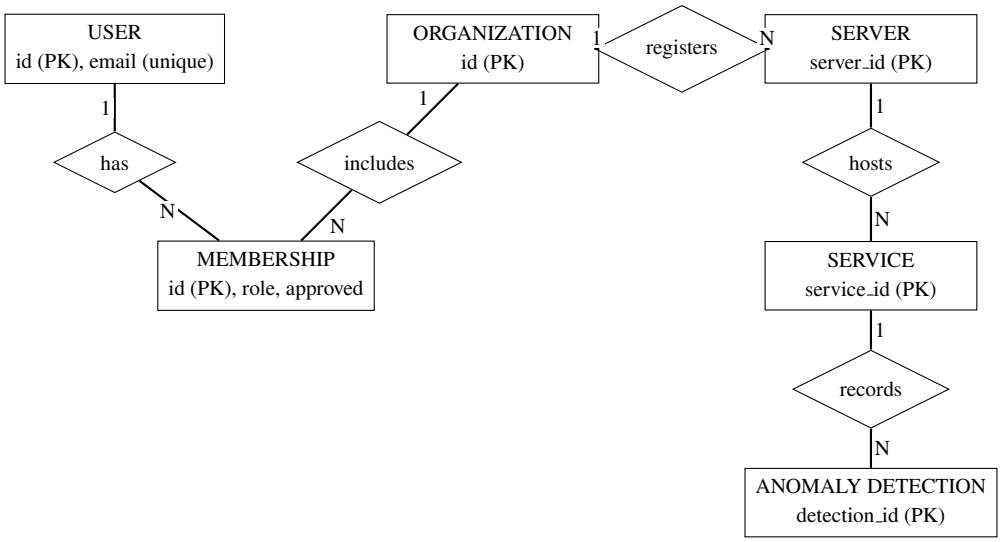


Figure 2.2: Core Infra Monitor ER model, redrawn in academic notation from backend models

2.3.2 Use Cases

The following concise use-case view was generated for this report from the organization, server, incident, and assistant routes. It summarizes role interactions and is not a replacement for the complete route documentation (`backend/infra_monitor/organization_urls.py`; `backend/incident/urls.py`; `backend/ai/urls.py`).

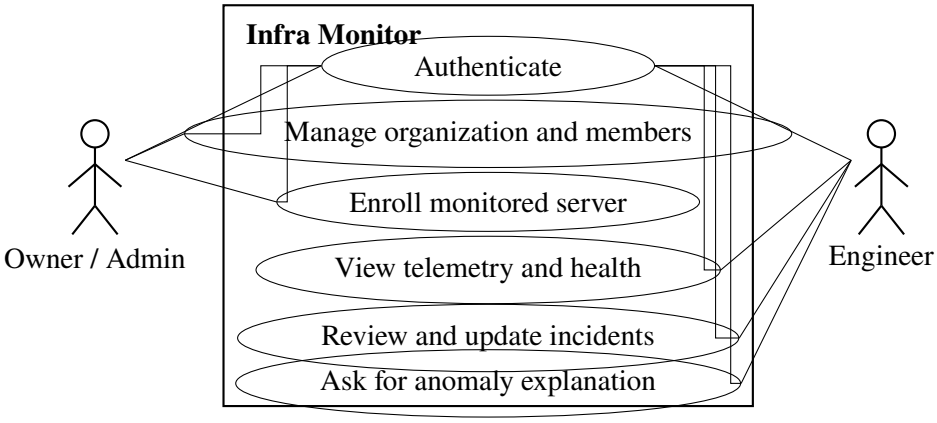


Figure 2.3: Report-generated use-case summary from current application routes

2.4 Database Schema

The application separates identity and organization membership from monitoring objects. A user can participate in organizations through a membership with role and approval state; an organization owns servers; a server contains services; and anomaly records reference the organization, server, and optionally a service. Alerts and incidents are separate operational records with organization and server/service references (`backend/accounts/models/organization_`

membership.py; backend/alert/models/alert.py; backend/incident/models/incident.py).

Table 2.2: Selected domain records and key relationships

Record	Key fields	Relationship / constraint
Organization	UUID primary key, name	Parent scope for servers, memberships, alerts, incidents, and detections.
OrganizationMembership	UUID key, user FK, organization FK, role, approved	Unique per user and organization; approved privileged roles are constrained.
Server	UUID key, organization FK, host name, environment, status	Host name is unique within an organization.
Service	UUID key, server FK, service name, status	Service name is unique within a server.
AnomalyDetection	UUID key, organization/server/service FKs, scores, feature values, window	Model-version and time-window uniqueness per service.
Alert / Incident	UUID keys, organization and optional server/service references	Lifecycle state, severity, and assignment fields support operations.

2.5 User Interface Design

The supplied Flutter screenshots show a dashboard overview, server inventory, metric detail, and incident queue. They are illustrative interface captures, not evidence of a formal user-acceptance test or measured production state. The screen composition is implemented in the Flutter client under frontend/lib/.

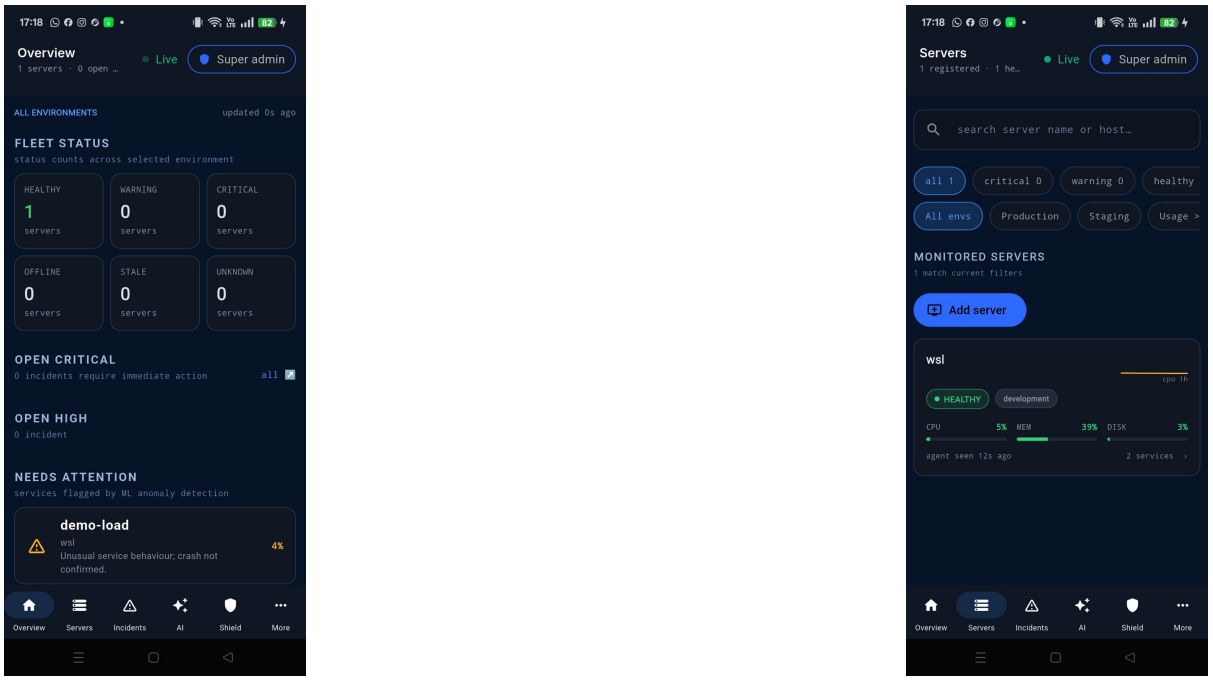


Figure 2.4: Dashboard overview (left) and monitored server list (right)

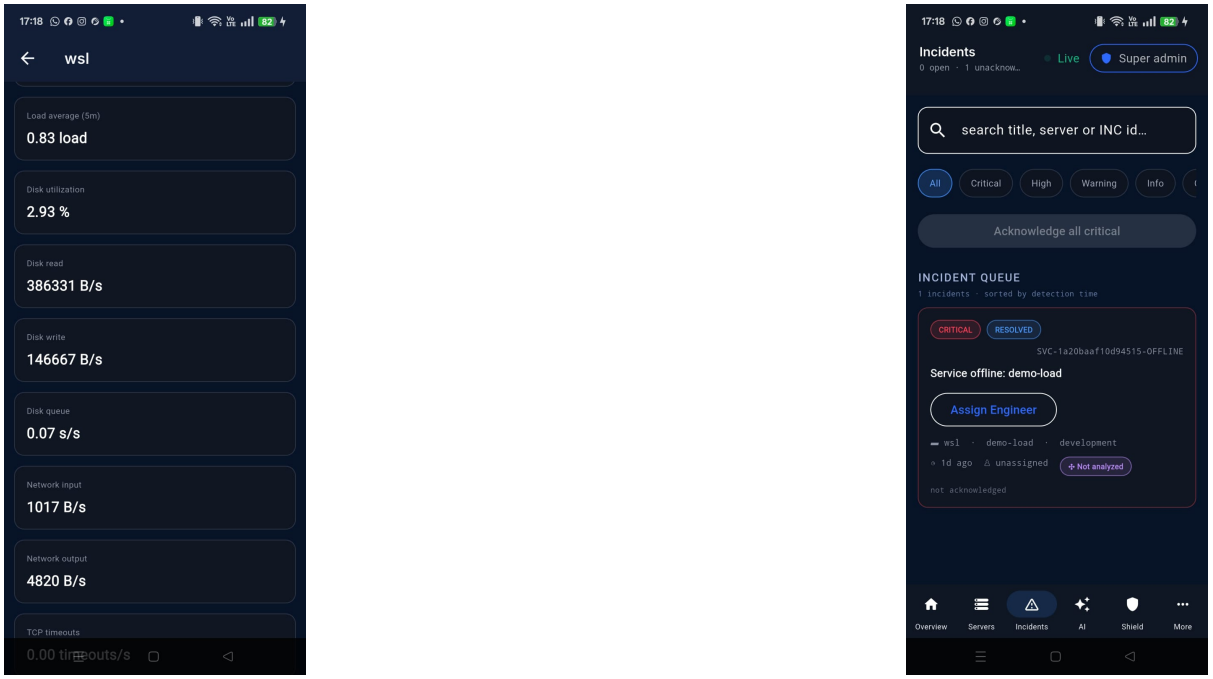


Figure 2.5: Service metric detail (left) and incident queue (right)

Chapter 3

Monitoring and Intelligent Features

3.1 Telemetry Collection

An organization owner or administrator starts server enrollment from the client. Django issues an enrollment command; the Linux host runs the installer and starts Grafana Alloy. The collector forwards Prometheus remote-write data to the Django ingestion path, which applies trusted organization/server identity before writing samples to VictoriaMetrics. Operational API reads query that time-series store rather than the separate PostgreSQL metric model ([3]; `backend/installer/`; `backend/servers/services/`).

PostgreSQL and VictoriaMetrics have distinct responsibilities: the former stores control-plane objects such as organizations, memberships, servers, and enrollment state, while the latter stores metric samples and labels (`docs/monitoring-end-to-end.md`; `docker-compose.yml`). This separation allows the application to associate time-series queries with the correct organization and monitored host.

3.2 Anomaly Detection and Request Shield

The ML service exposes separate endpoints for anomaly model training and inference, plus batch HTTP request classification (`model/app/main.py`). Celery tasks collect a completed service metric window, request training when a service model is absent and enough varied telemetry exists, then submit inference. The detection result is returned to Django for persistence (`backend/ml_model/tasks.py`; `model/app/pipeline/infer.py`).

Request Shield validates a fixed feature schema and classifies request feature vectors into GREEN, GRAY, or RED zones. The classifier implementation uses a Random Forest training function and produces a zone and confidence value; the repository does not establish a final accuracy result in this report (`backend/sanitize/features.py`; `model/app/pipeline/request_classifier.py`).

3.3 Advisory Assistant

The assistant is a separate conversational feature. Django prepares a structured anomaly evidence snapshot and streams a Gemini response. Its system instruction treats the anomaly as evidence of unusual behavior, not proof of a crash, and tells the model not to execute commands or change monitoring state (`backend/ai/services.py`; `backend/ai/consumers.py`). Thus, model output is advisory; the service lifecycle remains the application's authoritative availability state.

3.4 Operational Workflows

The interface presents health, metrics, alerts, and incidents together so an operator can move from a monitored resource to an operational event. The backend provides organization-scoped endpoints for servers, alerts, incidents, and anomaly detections. Incident workflows include acknowledgement, assignment, status, updates, evidence, and feedback; the routes are declared in `backend/incident/urls.py`. This chapter describes implemented paths and does not claim that a specific operational event was resolved by the system.

Chapter 4

Implementation Details

4.1 Application Modules

The backend is organized around domain packages rather than a single monolithic views module. `backend/accounts/` handles identity, organization and membership workflows; `backend/servers/` handles host/service records and metric queries; `backend/installer/` implements enrollment and telemetry ingestion; and `backend/alert/`, `backend/incident/`, `backend/ml_model/`, and `backend/ai/` handle event lifecycles and intelligent features ([1]; [5]).

The Flutter application is configured in `frontend/pubspec.yaml`. Domain URL modules are composed under `backend/infra_monitor/organization_urls.py`, while authentication, monitoring, and internal service routes are composed in `backend/infra_monitor/urls.py`.

4.2 Selected API Surface

The organization-scoped API prefix is `/api/organizations/{organization_id}/`. The following routes are defined in backend URL modules; this table summarizes the path shape and operation family rather than duplicating every request/response schema.

Table 4.1: Representative API routes from Django URL configuration

Route	Methods	Purpose
`/api/auth/register/`, `/api/auth/login/`	POST	Account registration and authentication.
`/api/organizations/{id}/servers/`	GET	List organization servers.
`/api/organizations/{id}/servers/{server_id}/metrics/`	GET	Query a server metric range.
`/api/organizations/{id}/alerts/`	GET	List organization alerts.
`/api/organizations/{id}/incidents/{incident_id}/assignment/`	POST	Change incident assignment.
`/api/organizations/{id}/anomalies/`	GET	List organization anomaly detections.
`/api/monitoring/` and `/api/internal/monitoring/`	Multiple	Enrollment and internal collector operations.

Route declarations can be checked in `backend/accounts/urls.py`, `backend/servers/urls.py`, `backend/alert/urls.py`, `backend/incident/urls.py`, and `backend/ml_model/urls.py`. User-facing organization APIs are scoped by organization and use the application's

authentication and authorization code; internal collector and ML routes use service credentials or enrollment credentials as documented in `docs/monitoring-end-to-end.md`.

4.3 Security and Access Control

The user model uses email as its login identifier and requires unique email addresses (`backend/accounts/models/users.py`). Organization membership stores OWNER, ADMIN, or ENGINEER roles and an approval state. Model constraints enforce uniqueness of a user's membership in an organization and approval for privileged roles (`backend/accounts/models/organization_membership.py`). Sensitive internal configuration is supplied through environment files, and the repository's setup guide warns against committing those files (`README.md`).

4.4 Local Execution and Verification

The root Compose file defines the local backend, ML service, PostgreSQL, Redis/Celery, and VictoriaMetrics services ([2]). The README documents setup and these backend quality checks: `python manage.py check`, `python manage.py makemigrations --check --dry-run`, `python manage.py test --noinput`, `ruff check .`, and `ruff format --check .` ([1]). Test files exist in several backend apps and in `frontend/test/`. No test execution or coverage run was performed for this report, so pass counts, performance figures, and coverage percentages are intentionally not reported.

Chapter 5

Conclusion

Infra Monitor combines organization-scoped infrastructure management with Linux host telemetry, operational event workflows, and machine-learning features. Its current implementation separates relational operational data from time-series samples, exposes a Flutter client and Django APIs, and orchestrates background and model services through the local Compose configuration (`docker-compose.yml`; `backend/infra_monitor/organization_urls.py`).

The repository contains implementation and test code, but this report does not assert production deployment, measured availability, classifier accuracy, or completed user-acceptance testing. These claims require repeatable test runs and recorded results. The supplied architecture illustration also omits some current AI paths; this is called out in Chapter 2 rather than silently treated as complete.

Future Work

Priorities for a subsequent iteration include updating the architecture illustration to show all AI integrations, executing and recording the documented backend and Flutter checks, adding reproducible performance and volume tests, and preparing a deployment and monitoring procedure for the target environment. Semester, submission date, and any project-management records remain for the team to complete.

Bibliography

[1] Infra Monitor project repository, *README.md*; project overview, setup, and quality-check commands.

[2] Infra Monitor project repository, *docker-compose.yml*; service definitions and local deployment configuration.

[3] Infra Monitor project repository, *docs/monitoring-end-to-end.md*; monitoring enrollment and telemetry data flow.

[4] Infra Monitor project repository, *model/app/main.py* and *model/app/pipeline/*; ML service interfaces and inference implementation.

[5] Infra Monitor project repository, backend source modules under *backend/*; routes, domain models, and application behavior.

[6] Infra Monitor project repository, *frontend/pubspec.yaml* and *frontend/lib/*; Flutter client dependencies and screens.